🖥️ Mémo PowerShell — BTS SIO option SISR (2e année)

> **Astuce de survie** : `Get-Help <commande>` et `Get-Command *motclé*` sont tes meilleurs amis. On ne retient pas tout par cœur, on sait **retrouver** ! Compléter en auto : `Tab` (nom de commande/paramètre), `↑` `↓` (historique), `F7` (liste historique).

---

1. 🧰 Les bases de la console

Commande (alias)	Rôle
`Get-Help` (`help`)	Affiche l'aide d'une commande. `-Examples` pour des exemples, `-Online` pour la doc web
`Get-Command` (`gcm`)	Trouve une commande (`Get-Command *firewall*`)
`Get-Member` (`gm`)	Affiche les propriétés/méthodes d'un objet
`Update-Help`	Met à jour les fichiers d'aide
`Clear-Host` (`cls`)	Efface l'écran
`Get-Date`	Date et heure système
`Get-PSDrive`	Liste les lecteurs (C:, D:, HKLM:, etc.)
`Set-ExecutionPolicy RemoteSigned`	Autorise les scripts locaux signés (à connaître pour les .ps1 !)

Les "verbes" à reconnaître

`Get` (lire) · `Set` (définir) · `New` (créer) · `Add` (ajouter) · `Remove` (supprimer) · `Set-Item` (modifier) · `Start/Stop` (démarrer/arrêter) · `Enable/Disable` (activer/désactiver)

---

2. 📂 Fichiers et dossiers

Commande (alias)	Rôle
`Get-ChildItem` (`ls`, `dir`)	Liste le contenu d'un dossier. `-Recurse` pour les sous-dossiers, `-Filter *.log` pour filtrer
`Set-Location` (`cd`)	Change de dossier
`Get-Location` (`pwd`)	Affiche le dossier courant
`New-Item -ItemType Directory` (`mkdir`)	Crée un dossier (`New-Item -ItemType File` pour un fichier)
`Copy-Item` (`cp`)	Copie (`-Recurse` pour un dossier)
`Move-Item` (`mv`)	Déplace / renomme
`Remove-Item` (`rm`)	Supprime (`-Recurse -Force` pour un dossier non vide)
`Get-Content` (`cat`)	Affiche le contenu d'un fichier

Commande (alias)	Rôle
`Set-Content` / `Add-Content`	Écrit / ajoute du texte dans un fichier
`Get-Item`	Propriétés d'un fichier (taille, dates…)
`Test-Path`	Vérifie qu'un chemin existe (retourne `$true` / `$false`)

---

3. 🔍 Le pipeline et les filtres (le cœur de PowerShell !)

PowerShell transmet des **objets**, pas du texte. On enchaîne avec `|` :

```
Get-Process | Where-Object { $_.CPU -gt 100 } | Sort-Object
CPU -Descending | Select-Object -First 5 Name, CPU
```

Commande (alias)	Rôle
`Where-Object` (`?`)	Filtre les objets : `{ $_.Status -eq "Running" }`
`Sort-Object` (`sort`)	Trie (`-Descending` pour décroissant)
`Select-Object` (`select`)	Choisit des propriétés, ou `-First 10` / `-Last 5`
`Measure-Object`	Compte / moyenne / somme (`Count`, `Sum`, `Average`)
`Group-Object`	Regroupe par valeur
`ForEach-Object` (`%`)	Traite chaque objet du pipeline
`Out-File`	Envoie vers un fichier (alternative : `Export-Csv`, `Export-Clixml`)
`Format-Table` (`ft`) / `Format-List` (`fl`)	Change l'affichage (jamais avant de filtrer !)

Opérateurs de comparaison (à connaître par cœur)

`-eq` égal · `-ne` différent · `-gt` supérieur · `-lt` inférieur · `-ge` / `-le` · `-like` avec jokers (`*`) · `-match` regex · `-and` / `-or` / `-not`

---

4. 🧮 Variables, scripts et logique

```
$nom = "SRV-AD01" # Variable
$serv = Get-Service Spooler # Un objet dans une variable
$nom.ToUpper() # Méthode
"Le service $($serv.Name) est $($serv.Status)" # Chaîne interpolée
# Condition
if ($serv.Status -eq "Running") { "OK" } else { "KO" }
# Boucle
foreach ($f in Get-ChildItem C:\Logs) { $f.Name }
# Fonction
function Get-EspaceLibre {
```

```
  param([string]$Lettre = "C:")
  Get-PSDrive $Lettre | Select-Object Name, Free
}
```

Autres bases : `$args`, `Read-Host "Question"` (saisie utilisateur), `Write-Host` / `Write-Output`, `1..10` (plage), tableaux `$t = 1,2,3`, tables de hachage `$h = @{cle="valeur"}`.

---

5. ⚙️ Système et processus

Commande	Rôle
`Get-Process` (`ps`)	Liste les processus (`-Name notepad`)
`Stop-Process -Name notepad -Force`	Tue un processus
`Start-Process notepad`	Lance un programme (`-Verb RunAs` pour admin)
`Get-Service`	Liste les services (`-Name wuauserv`)
`Restart-Service`, `Start-Service`, `Stop-Service`	Gère un service
`Set-Service -Name wuauserv -StartupType Automatic`	Change le type de démarrage
`Get-ComputerInfo`	Infos système complètes
`Get-EventLog -LogName System -Newest 20`	Journaux d'événements (ou `Get-WinEvent`)
`Restart-Computer` / `Stop-Computer`	Redémarre / éteint (`-ComputerName` pour à distance)
`Get-HotFix`	Liste les mises à jour installées
`Get-ScheduledTask`	Tâches planifiées

---

6. 💾 Disques, stockage et réseau

Commande	Rôle
`Get-Disk` / `Get-Partition` / `Get-Volume`	Inventaire disques, partitions, volumes
`Initialize-Disk`, `New-Partition`, `Format-Volume`	Prépare et formate un disque ⚠️ destructif
`Get-NetIPAddress` / `Get-NetIPConfiguration`	Configuration IP
`New-NetIPAddress -InterfaceIndex 12 -IPAddress 192.168.1.10 -PrefixLength 24 -DefaultGateway 192.168.1.1`	Attribue une IP statique
`Set-DnsClientServerAddress -InterfaceIndex 12 -ServerAddresses 192.168.1.1`	Configure le DNS
`Get-NetAdapter`	Cartes réseau (`Enable-/Disable-NetAdapter`)
`Test-Connection` (`ping`) / `Test-NetConnection`	Test de connectivité (`-Port 443` pour un port)

Commande	Rôle
`Resolve-DnsName www.google.fr`	Résolution DNS (`nslookup` like)
`Get-NetTCPConnection`	Connexions/ports en écoute (`-State Listen`)
`Get-NetFirewallRule`	Règles du pare-feu
`Get-SmbShare` / `New-SmbShare`	Partages Windows
`Get-WindowsFeature` / `Install-WindowsFeature`	Rôles et fonctionnalités **Windows Server** ⭐

---

7. 👥 Utilisateurs et Active Directory (Windows Server)

```
# Modules AD : Import-Module ActiveDirectory (à installer via Install-
WindowsFeature RSAT-AD-PowerShell)
New-ADUser -Name "Marie Dupont" -SamAccountName mdupont `
 -UserPrincipalName
mdupont@monlab.local -AccountPassword (Read-Host -AsSecureString "MDP") `
 -Enabled $true -Path "OU=Stagiaires,DC=monlab,DC=local"
Get-ADUser -Filter * -SearchBase "OU=Stagiaires,DC=monlab,DC=local" # Lister
Enable-ADAccount / Disable-ADAccount -Identity mdupont # Activer / désactiver
Set-ADAccountPassword -Identity mdupont -Reset # Réinitialiser le mot de passe
Add-ADGroupMember -Identity "G-Stagiaires" -Members mdupont # Ajouter à un groupe
New-ADOrganizationalUnit -Name "Stagiaires" -Path "DC=monlab,DC=local" # Créer une
UO
Get-ADGroupMember -Identity "G-Stagiaires" # Membres d'un groupe
Get-ADComputer -Filter * # Machines du domaine
```

Localement (hors AD) : `Get-LocalUser`, `New-LocalUser`, `Get-LocalGroup`, `Add-LocalGroupMember -Group "Administrateurs" -Member "Marie"`.

---

8. 🌐 Administration à distance et hyper-v

Commande	Rôle
`Enter-PSSession -ComputerName SRV01 -Credential (Get-Credential)`	Session distante interactive (WinRM) ⭐
`Invoke-Command -ComputerName SRV01 -ScriptBlock { hostname }`	Exécute une commande à distance (sur plusieurs serveurs à la fois !)
`New-PSSession` / `Get-PSSession` / `Remove-PSSession`	Sessions persistantes
`Get-VM` / `Start-VM` / `Stop-VM`	Machines virtuelles Hyper-V
`New-VM -Name "TEST01" -MemoryStartupBytes 2GB -NewVHDPath C:\VM\test.vhdx -NewVHDSizeBytes 40GB`	Crée une VM
	Snapshot

Commande	Rôle
`Checkpoint-VM -Name "TEST01" -SnapshotName "Avant-manip"`	
`Copy-Item -ToSession $s`	Copie de fichiers via une session distante

---

9. 📊 Rapports et exports (pour les rendus de TP !)

```
Get-Service | Where-Object Status -eq "Running" | Export-Csv C:
\Rapports\services.csv -NoTypeInformation -Delimiter ";"
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 | Out-File C:
\Rapports\top10.txt
Get-EventLog System -EntryType Error -Newest 50 | Format-Table -AutoSize
```

---

10. ⚠️ Les pièges classiques (et comment les éviter)

- **Exécution de script bloquée** : `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`, puis `.\monscript.ps1` (pas `monscript.ps1` tout court).
- **Comparer des chaînes** : `-eq` et non `==` (qui existe mais marche différemment).
- `Format-Table` **casse le pipeline** : filtre/trie d'abord, formate à la fin.
- `-WhatIf` : ajoute-le à `Remove-Item`, `Stop-Service`, etc. pour simuler sans risque. Ton meilleur ami en prod ! 👌
- **Majuscule/minuscule** : PowerShell n'est pas sensible à la casse (`-ceq` pour la comparaison stricte).
- `$_` : c'est l'objet courant du pipeline, pas une faute de frappe.

🎯 Pour réviser

1. Refais les 3 commandes "réflexe" : `Get-Help`, `Get-Command`, `Get-Member`.
2. Entraîne-toi à une chaîne complète : *lister les services arrêtés automatiques* → `Get-Service | Where-Object { $_.StartType -eq "Automatic" -and $_.Status -ne "Running" }`.
3. Sur Windows Server : entraîne-toi à `Install-WindowsFeature`, `New-ADUser`, `Enter-PSSession` — les grands classiques des épreuves !

Bon courage, tu vas poser PowerShell comme un pro ! 💪